

Linear Regression Model For Breast Cancer

CCDS211: Introduction To Data science

Distribution of duties

Name	ID	Task
Mawadah Abdulrahim	2311104	All Members works together
Tala Saleem	2310919	
Ghala Mohammed	2317323	
Dalia Alqahtani	2310316	

Load Data:

```
> # Load the libraries after installation
> library(data.table)
>
> # Load the dataset
> data <- fread("C:/Users/dell/Documents/machine/data.csv",fill=TRUE) # Replace "data.csv" with the actual path to your file
> # Data Exploration
> print(head(data))
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean
	<int>	<char>	<num>	<num>	<num>	<num>	<num>	<num>
1:	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760
2:	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864
3:	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990
4:	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390
5:	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280
6:	843786	M	12.45	15.70	82.57	477.1	0.12780	0.17000

```
> print(head(data))
```

	concavity_mean	concave points_mean	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	perimeter_se
	<num>	<num>	<num>	<num>	<num>	<num>	<num>
1:	0.3001	0.14710	0.2419	0.07871	1.0950	0.9053	8.589
2:	0.0869	0.07017	0.1812	0.05667	0.5435	0.7339	3.398
3:	0.1974	0.12790	0.2069	0.05999	0.7456	0.7869	4.585
4:	0.2414	0.10520	0.2597	0.09744	0.4956	1.1560	3.445
5:	0.1980	0.10430	0.1809	0.05883	0.7572	0.7813	5.438
6:	0.1578	0.08089	0.2087	0.07613	0.3345	0.8902	2.217

```
> print(head(data))
```

	area_se	smoothness_se	compactness_se	concavity_se	concave points_se	symmetry_se	fractal_dimension_se
	<num>	<num>	<num>	<num>	<num>	<num>	<num>
1:	153.40	0.006399	0.04904	0.05373	0.01587	0.03003	0.006193
2:	74.08	0.005225	0.01308	0.01860	0.01340	0.01389	0.003532
3:	94.03	0.006150	0.04006	0.03832	0.02058	0.02250	0.004571
4:	27.23	0.009110	0.07458	0.05661	0.01867	0.05963	0.009208
5:	94.44	0.011490	0.02461	0.05688	0.01885	0.01756	0.005115
6:	27.19	0.007510	0.03345	0.03672	0.01137	0.02165	0.005082

```
> print(head(data))
```

	radius_worst	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst
	<num>	<num>	<num>	<num>	<num>	<num>	<num>
1:	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119
2:	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416
3:	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504
4:	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869
5:	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000
6:	15.47	23.75	103.40	741.6	0.1791	0.5249	0.5355

```
> print(head(data))
```

	concave points_worst	symmetry_worst	fractal_dimension_worst	v33
	<num>	<num>	<num>	<logi>
1:	0.2654	0.4601	0.11890	NA
2:	0.1860	0.2750	0.08902	NA
3:	0.2430	0.3613	0.08758	NA
4:	0.2575	0.6638	0.17300	NA
5:	0.1625	0.2364	0.07678	NA
6:	0.1741	0.3985	0.12440	NA

```
> print(str(data))
```

Classes 'data.table' and 'data.frame': 569 obs. of 33 variables:

```
$ id          : int  842302 842517 84300903 84348301 84358402 843786 844359 84458202 844981 84501001 ...
$ diagnosis   : chr   "M" "M" "M" "M" ...
$ radius_mean : num   18 20.6 19.7 11.4 20.3 ...
$ texture_mean : num   10.4 17.8 21.2 20.4 14.3 ...
$ perimeter_mean : num  122.8 132.9 130 77.6 135.1 ...
$ area_mean    : num  1001 1326 1203 386 1297 ...
$ smoothness_mean : num  0.1184 0.0847 0.1096 0.1425 0.1003 ...
$ compactness_mean : num  0.2776 0.0786 0.1599 0.2839 0.1328 ...
$ concavity_mean : num  0.3001 0.0869 0.1974 0.2414 0.198 ...
$ concave_points_mean : num  0.1471 0.0702 0.1279 0.1052 0.1043 ...
$ symmetry_mean : num  0.242 0.181 0.207 0.26 0.181 ...
$ fractal_dimension_mean : num  0.0787 0.0567 0.06 0.0974 0.0588 ...
$ radius_se    : num  1.095 0.543 0.746 0.496 0.757 ...
$ texture_se    : num  0.905 0.734 0.787 1.156 0.781 ...
$ perimeter_se  : num  8.59 3.4 4.58 3.44 5.44 ...
$ area_se       : num  153.4 74.1 94 27.2 94.4 ...
$ smoothness_se : num  0.0064 0.00522 0.00615 0.00911 0.01149 ...
$ compactness_se : num  0.049 0.0131 0.0401 0.0746 0.0246 ...
$ concavity_se  : num  0.0537 0.0186 0.0383 0.0566 0.0569 ...
$ compactness_se : num  0.049 0.0131 0.0401 0.0746 0.0246 ...
$ concavity_se  : num  0.0537 0.0186 0.0383 0.0566 0.0569 ...
$ concave_points_se : num  0.0159 0.0134 0.0206 0.0187 0.0188 ...
$ symmetry_se    : num  0.03 0.0139 0.0225 0.0596 0.0176 ...
$ fractal_dimension_se : num  0.00619 0.00353 0.00457 0.00921 0.00511 ...
$ radius_worst   : num  25.4 25 23.6 14.9 22.5 ...
$ texture_worst  : num  17.3 23.4 25.5 26.5 16.7 ...
$ perimeter_worst : num  184.6 158.8 152.5 98.9 152.2 ...
$ area_worst     : num  2019 1956 1709 568 1575 ...
$ smoothness_worst : num  0.162 0.124 0.144 0.21 0.137 ...
$ compactness_worst : num  0.666 0.187 0.424 0.866 0.205 ...
$ concavity_worst : num  0.712 0.242 0.45 0.687 0.4 ...
$ concave_points_worst : num  0.265 0.186 0.243 0.258 0.163 ...
$ symmetry_worst  : num  0.46 0.275 0.361 0.664 0.236 ...
$ fractal_dimension_worst : num  0.1189 0.089 0.0876 0.173 0.0768 ...
$ v33            : logi  NA NA NA NA NA ...
- attr(*, "internal.selfref")=<externalptr>
NULL
```

Dependent & Independent Variables:

For this analysis, we chose a linear regression model.

The independent variable is `radius_mean`, which represents the mean radius of the tumor.

The dependent variable is `texture_mean`, which represents the mean texture of the tumor.

The reason for this choice is that `radius_mean` is an important feature that can affect `texture_mean`, and using a linear regression model helps us understand how changes in the independent variable affect the dependent variable.

```
> # Split the data into features (X) and target (y)
> X <- data[, c("radius_mean"), with = FALSE] # Independent variable (radius_mean)
> y <- data$texture_mean # Dependent variable (texture_mean)
>
```

In this dataset, there are no null values in the columns we are using (`radius_mean` and `texture_mean`). However, if there are missing values (null values), we can use the `na.omit()` function to remove any rows with missing data. Alternatively, we can use techniques such as imputation (for example, replacing missing values with the mean or median) or prediction models to estimate missing values.

```
> # Handle missing values (if any) by replacing them with the mean
> data$radius_mean[is.na(data$radius_mean)] <- mean(data$radius_mean, na.rm = TRUE)
> data$texture_mean[is.na(data$texture_mean)] <- mean(data$texture_mean, na.rm = TRUE)
>
> # Ensure no rows with NA values remain
> data <- na.omit(data)
>
```

Since the dataset did not contain any null values, I did not need to use `na.omit()`. In case of missing values, I would use the mean or `na.omit()` for simplicity, especially in cases where the rows with missing data are few.

Data summary:

```
Console Terminal Background Jobs
R v4.4.2
> summary(data)
      id      diagnosis      radius_mean      texture_mean      perimeter_mean      area_mean
Min.   : 8670      Length:569      Min.   : 6.981      Min.   : 9.71      Min.   : 43.79      Min.   : 143.5
1st Qu.: 869218      Class :character      1st Qu.:11.700      1st Qu.:16.17      1st Qu.: 75.17      1st Qu.: 420.3
Median : 906024      Mode  :character      Median :13.370      Median :18.84      Median : 86.24      Median : 551.1
Mean   : 30371831      Mean   :14.127      Mean   :19.29      Mean   : 91.97      Mean   : 654.9
3rd Qu.: 8813129      3rd Qu.:15.780      3rd Qu.:21.80      3rd Qu.:104.10      3rd Qu.: 782.7
Max.   :911320502      Max.   :28.110      Max.   :39.28      Max.   :188.50      Max.   :2501.0

smoothness_mean      compactness_mean      concavity_mean      concave points_mean      symmetry_mean      fractal_dimension_mean
Min.   :0.05263      Min.   :0.01938      Min.   :0.00000      Min.   :0.00000      Min.   :0.1060      Min.   :0.04996
1st Qu.:0.08637      1st Qu.:0.06492      1st Qu.:0.02956      1st Qu.:0.02031      1st Qu.:0.1619      1st Qu.:0.05770
Median :0.09587      Median :0.09263      Median :0.06154      Median :0.03350      Median :0.1792      Median :0.06154
Mean   :0.09636      Mean   :0.10434      Mean   :0.08880      Mean   :0.04892      Mean   :0.1812      Mean   :0.06280
3rd Qu.:0.10530      3rd Qu.:0.13040      3rd Qu.:0.13070      3rd Qu.:0.07400      3rd Qu.:0.1957      3rd Qu.:0.06612
Max.   :0.16340      Max.   :0.34540      Max.   :0.42680      Max.   :0.20120      Max.   :0.3040      Max.   :0.09744

radius_se      texture_se      perimeter_se      area_se      smoothness_se      compactness_se
Min.   :0.1115      Min.   :0.3602      Min.   :0.757      Min.   : 6.802      Min.   :0.001713      Min.   :0.002252
1st Qu.:0.2324      1st Qu.:0.8339      1st Qu.: 1.606      1st Qu.:17.850      1st Qu.:0.005169      1st Qu.:0.013080
Median :0.3242      Median :1.1080      Median : 2.287      Median :24.530      Median :0.006380      Median :0.020450
Mean   :0.4052      Mean   :1.2169      Mean   : 2.866      Mean   :40.337      Mean   :0.007041      Mean   :0.025478
3rd Qu.:0.4789      3rd Qu.:1.4740      3rd Qu.: 3.357      3rd Qu.:45.190      3rd Qu.:0.008146      3rd Qu.:0.032450
Max.   :2.8730      Max.   :4.8850      Max.   :21.980      Max.   :542.200      Max.   :0.031130      Max.   :0.135400

concavity_se      concave points_se      symmetry_se      fractal_dimension_se      radius_worst      texture_worst
Min.   :0.00000      Min.   :0.00000      Min.   :0.007882      Min.   :0.0008948      Min.   : 7.93      Min.   :12.02
1st Qu.:0.01509      1st Qu.:0.007638      1st Qu.:0.015160      1st Qu.:0.0022480      1st Qu.:13.01      1st Qu.:21.08
Median :0.02589      Median :0.010930      Median :0.018730      Median :0.0031870      Median :14.97      Median :25.41
Mean   :0.03189      Mean   :0.011796      Mean   :0.020542      Mean   :0.0037949      Mean   :16.27      Mean   :25.68
3rd Qu.:0.04205      3rd Qu.:0.014710      3rd Qu.:0.023480      3rd Qu.:0.0045580      3rd Qu.:18.79      3rd Qu.:29.72
Max.   :0.39600      Max.   :0.052790      Max.   :0.078950      Max.   :0.0298400      Max.   :36.04      Max.   :49.54

perimeter_worst      area_worst      smoothness_worst      compactness_worst      concavity_worst      concave points_worst
Min.   : 50.41      Min.   :185.2      Min.   :0.07117      Min.   :0.02729      Min.   :0.0000      Min.   :0.00000
1st Qu.: 84.11      1st Qu.:515.3      1st Qu.:0.11660      1st Qu.:0.14720      1st Qu.:0.1145      1st Qu.:0.06493
Median : 97.66      Median :686.5      Median :0.13130      Median :0.21190      Median :0.2267      Median :0.09993
Mean   :107.26      Mean   :880.6      Mean   :0.13237      Mean   :0.25427      Mean   :0.2722      Mean   :0.11461
3rd Qu.:125.40      3rd Qu.:1084.0      3rd Qu.:0.14600      3rd Qu.:0.33910      3rd Qu.:0.3829      3rd Qu.:0.16140
Max.   :251.20      Max.   :4254.0      Max.   :0.22260      Max.   :1.05800      Max.   :1.2520      Max.   :0.29100

symmetry_worst      fractal_dimension_worst      V33
Min.   :0.1565      Min.   :0.05504      Mode:logical
1st Qu.:0.2504      1st Qu.:0.07146      NA's:569
Median :0.2822      Median :0.08004
Mean   :0.2901      Mean   :0.08395
3rd Qu.:0.3179      3rd Qu.:0.09208
Max.   :0.6638      Max.   :0.20750
```

BUILDING MODEL AND SUMMARY:

```
>
> # Split the data into training and testing sets
> set.seed(123) # For reproducibility
> data_split <- sample(1:nrow(data), size = floor(0.8 * nrow(data))) # 80% for training set
> train_data <- data[data_split, ] # Training set
> test_data <- data[-data_split, ] # Testing set
>

>
> # Check if train_data and test_data contain sufficient rows
> if (nrow(train_data) == 0 || nrow(test_data) == 0) {
+   stop("Train or Test data is empty. Check data splitting logic or dataset integrity.")
+ }
>
> # Check for missing or invalid values in train_data
> print(summary(train_data$radius_mean))
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 6.981 11.650 13.200 14.002 15.510 28.110
```

▪ Checking for missing or invalid values in the training data:

- For the radius_mean variable in the training data:
 - Min: 6.981
 - Q1: 11.650
 - Median: 13.200
 - Average: 14.002
 - Q3: 15.510
 - Max: 28.110
- For the texture_mean variable in the training data:
 - Min: 9.71
 - Q1: 10.11
 - Median: 18.82
 - Average: 19.22
 - Q3: 21.10
 - Max: 33.56

These summaries show the range, central tendency, and spread of the data for both variables, confirming that the values are valid and reasonable.

- **Linear Regression Model Summary:**

- Model Formula: $\text{texture_mean} \sim \text{radius_mean}$

- Residuals:

Minimum Residual: -8.6383

Residuals in the first quartile: -2.8024

Mean Residual: -0.2753

Residuals in the third quartile: 2.3962

Maximum Residual: 13.7227

This shows the distribution of the residuals, which are the differences between the observed and expected values.

- Parameters:

Intercept: 13.54790 (Statistically significant with $p < 2e-16$)

This represents the expected texture_mean value when radius_mean is 0.

Slope (radius_mean): 0.40498 (Statistically significant with $p = 7.76e-14$)

For every unit increase in radius_mean, texture_mean increases by 0.40498 on average.

- Model fit measures:

Residual Standard Error (RSE): 3.92

This is the average distance that observed values move away from the regression line.

R-Squared Multiplier: 0.1161

11.61% of the variance in texture_mean is explained by radius_mean.

Adjusted R-Squared: 0.1142

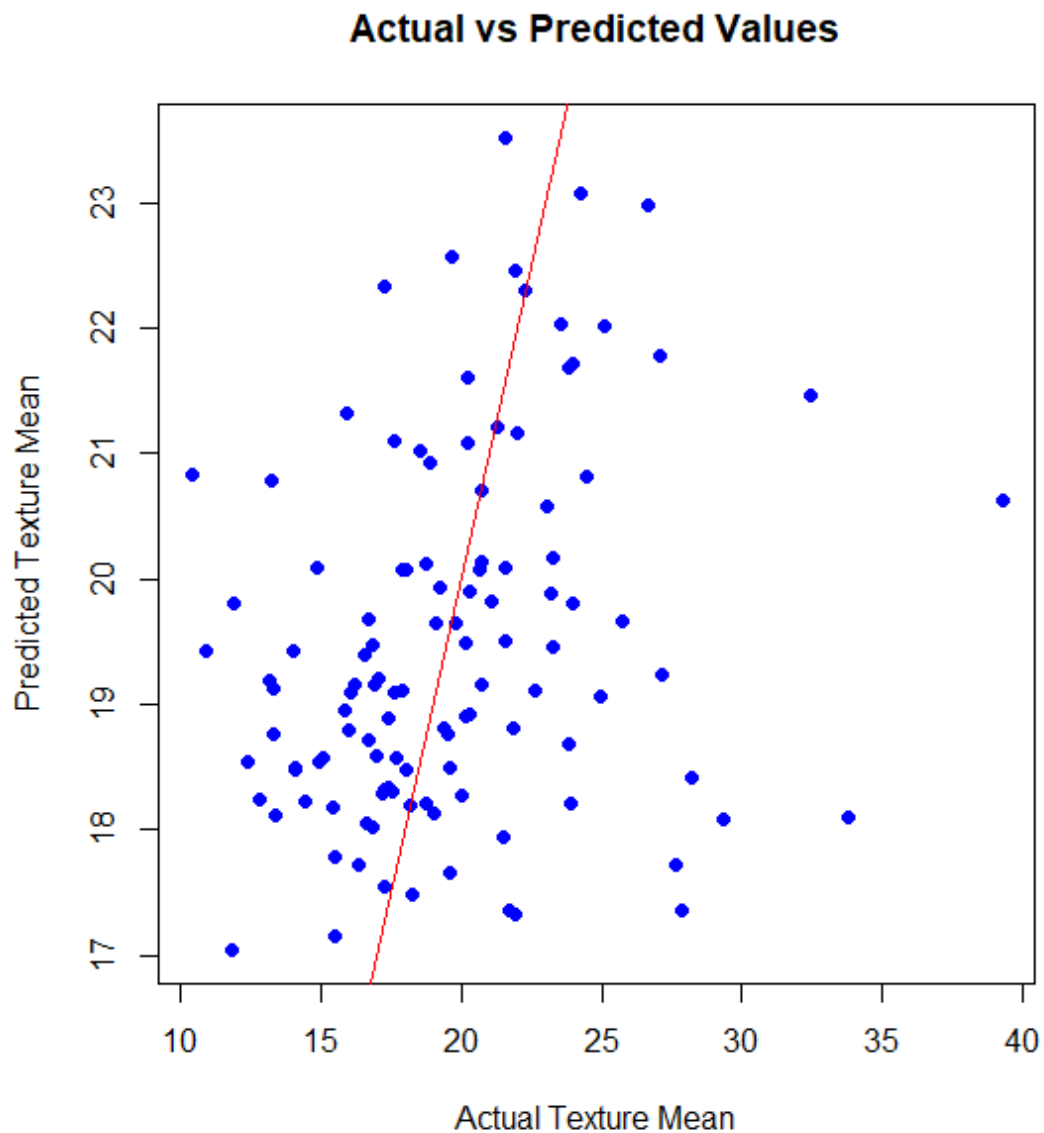
This measure is adjusted for the number of predictors in the model.

F-Statistic: 59.53

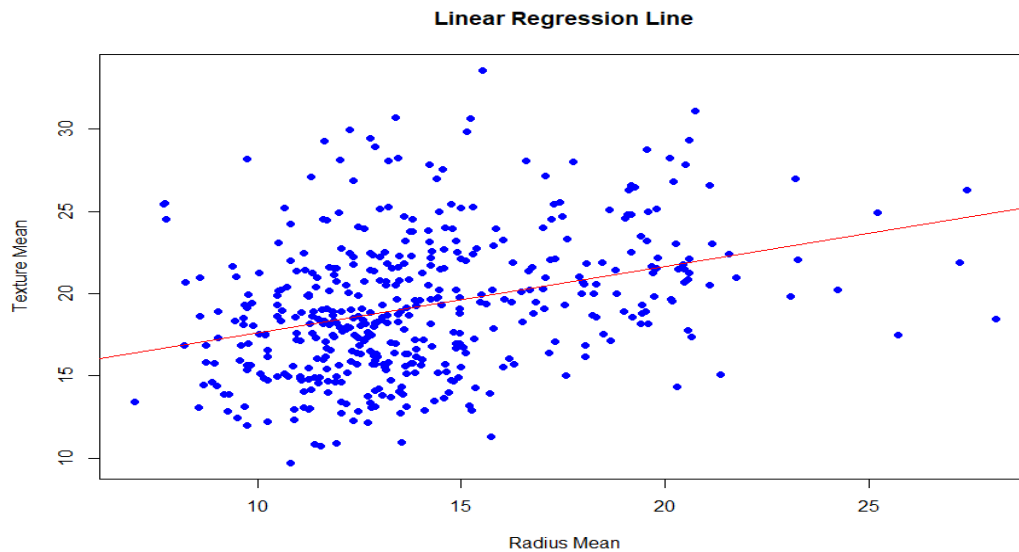
Indicates that the model is statistically significant with $p < 7.761e-14$.

CREATING PLOT(CHART):

```
> # Plot 1: Actual vs Predicted Values
> plot(test_data$texture_mean, predictions,
+       main = "Actual vs Predicted Values",
+       xlab = "Actual Texture Mean",
+       ylab = "Predicted Texture Mean",
+       pch = 19, col = "blue")
> abline(0, 1, col = "red") # Add a reference line
>
```



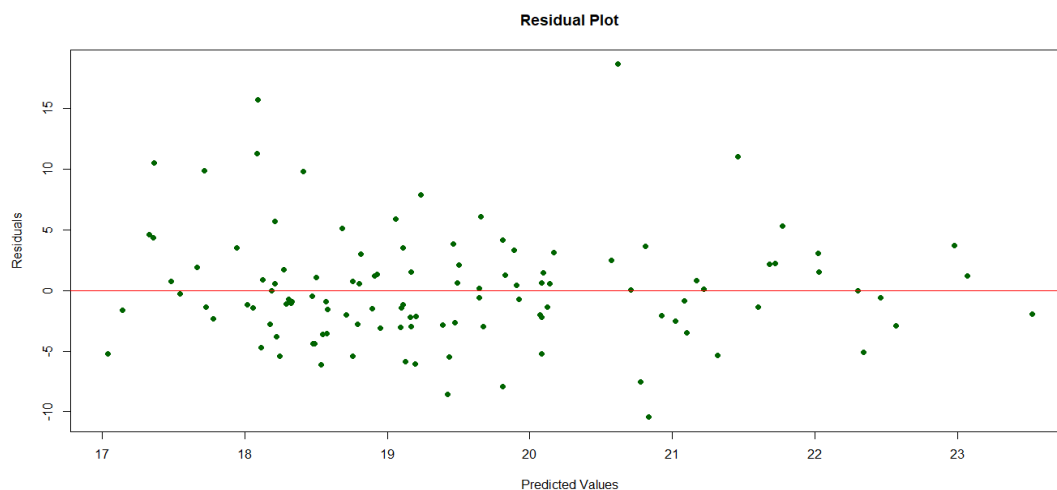
```
> # Plot 3: Linear Regression Line
> plot(train_data$radius_mean, train_data$texture_mean,
+       main = "Linear Regression Line",
+       xlab = "Radius Mean",
+       ylab = "Texture Mean",
+       pch = 19, col = "blue")
> abline(lm_model, col = "red") # Add the regression line
✓
```



```
> # Plot 2: Residual Plot
> residuals <- test_data$texture_mean - predictions
> plot(predictions, residuals,
+       main = "Residual Plot",
+       xlab = "Predicted values",
+       ylab = "Residuals",
+       pch = 19, col = "darkgreen")
> abline(h = 0, col = "red") # Add a horizontal reference line
```

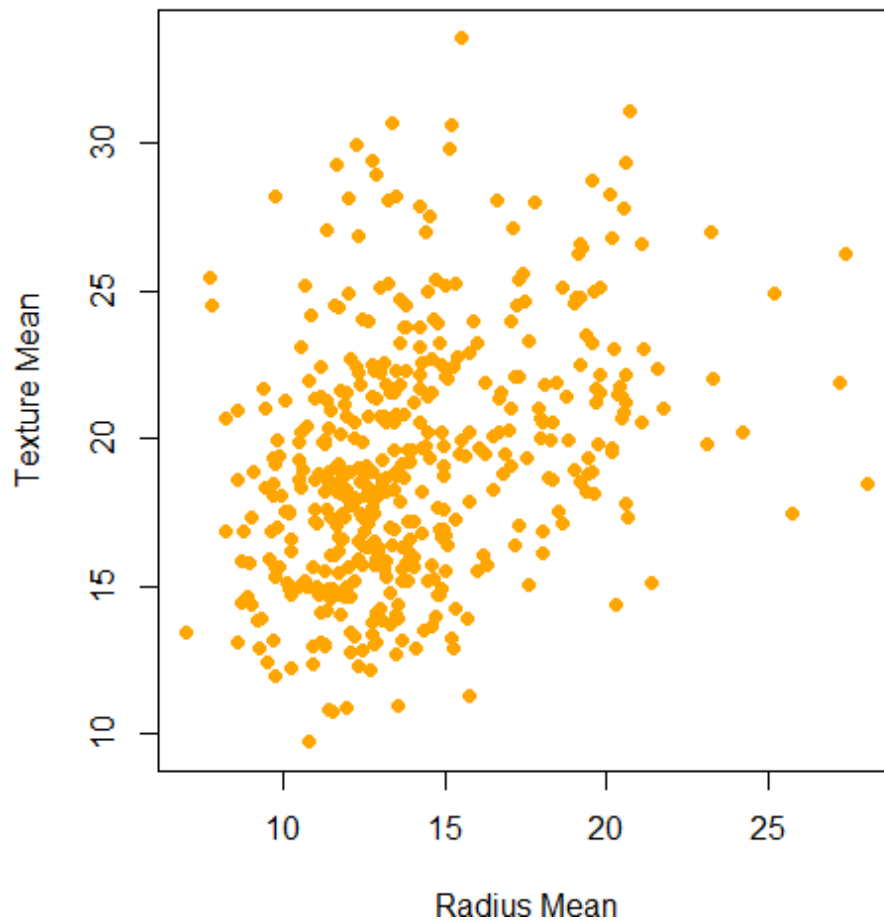
Plot Zoom

— □ ×



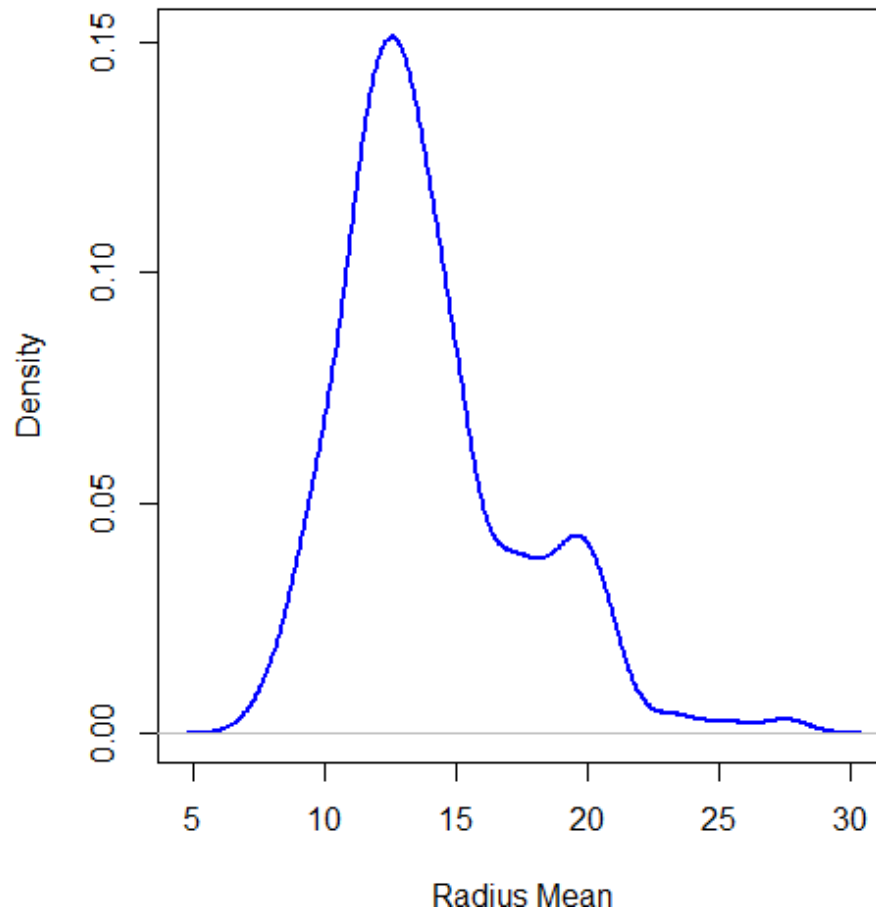
```
> # Plot 4: Scatter Plot
> plot(train_data$radius_mean, train_data$texture_mean,
+       main = "Scatter Plot of Radius Mean vs Texture Mean",
+       xlab = "Radius Mean",
+       ylab = "Texture Mean",
+       pch = 19, col = "orange")
✓
```

Scatter Plot of Radius Mean vs Texture Mean

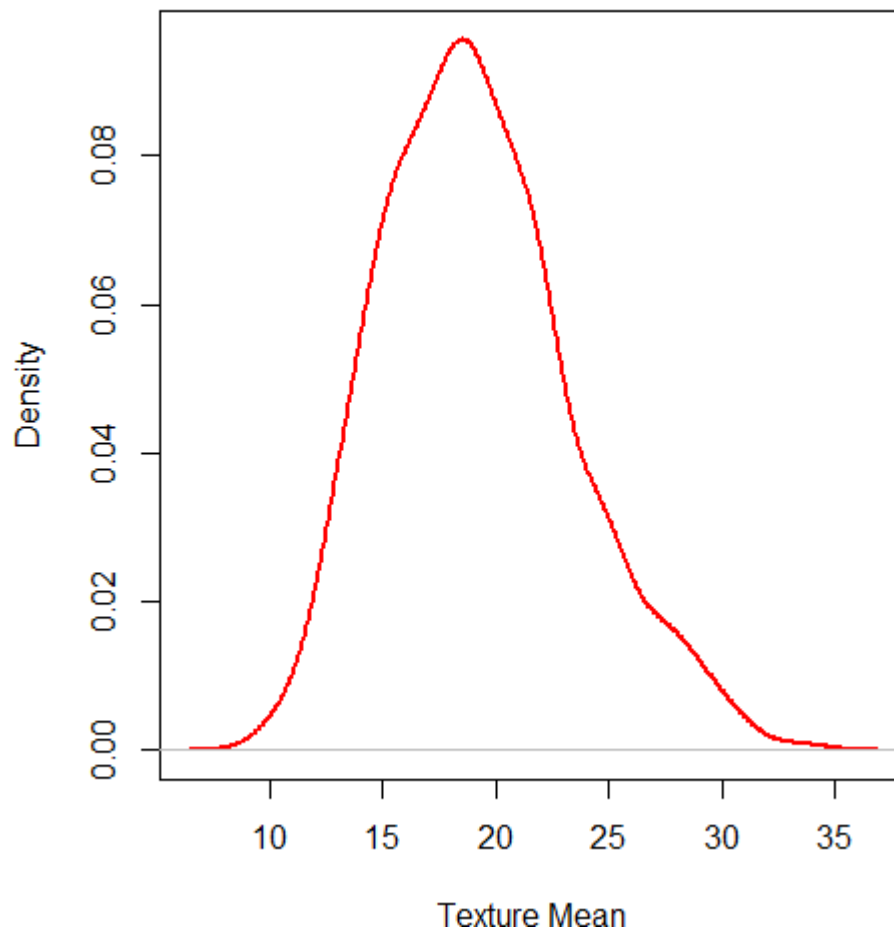


```
>
> # Plot 5: Density Plot for Radius Mean
> plot(density(train_data$radius_mean),
+      main = "Density Plot of Radius Mean",
+      xlab = "Radius Mean",
+      ylab = "Density",
+      col = "blue", lwd = 2)
>
> # Plot 6: Density Plot for Texture Mean
> plot(density(train_data$texture_mean),
+      main = "Density Plot of Texture Mean",
+      xlab = "Texture Mean",
+      ylab = "Density",
+      col = "red", lwd = 2)
> |
```

Density Plot of Radius Mean



Density Plot of Texture Mean



- **Explanation of all charts based on the results:**
- **Actual vs. Predicted Plot:**
 - Purpose:
 - This plot compares the actual texture_mean values from the test dataset to the values predicted by the linear regression model.
 - Notes: Ideally,
 - all points should line up along the red diagonal line, which represents perfect predictions.
 - In this case, there is some deviation from the line, indicating that while the model captures the overall trend, there are errors in the prediction.
 - This is consistent with the RMSE value (4.63), indicating that the model has moderate accuracy.

1. Residual Plot:

- Purpose:
 - This plot displays the residuals (differences between actual and predicted values) versus the predicted values.
- Notes:
 - The residuals are scattered around the horizontal line at 0, indicating that there is no clear pattern in the errors.
 - The random dispersion indicates that the model's assumptions (linearity and homoscedasticity) are likely valid.
 - However, some large residual values (outliers) indicate that the model may not capture all of the variance in texture_mean.

2. Linear Regression Line Plot:

- Purpose:
 - This plot shows the relationship between radius_mean and texture_mean in the training data, along with the fitted regression line.
- Notes:
 - The regression line shows a positive slope, consistent with the coefficient (0.40498) of the model.
 - The points are scattered around the line, indicating that radius_mean explains some of the variance in texture_mean, but not all of it.
 - This is supported by the R-squared value (0.1161), which shows that only 11.61% of the variance in texture_mean is explained by radius_mean.

3. Scatterplot of mean radius vs. mean texture:

- Purpose:
 - This plot shows the raw relationship between radius_mean and texture_mean in the training data.
- Notes:
 - The scatter plots indicate a weak positive correlation, which is consistent with a low R-squared value.
 - While there is an overall upward trend, the spread of the points suggests that additional variables may be influencing texture_mean.

4. Density plot of radius_mean:

- Purpose:
 - This plot shows the distribution of radius_mean values in the training dataset.
- Notes:
 - The density curve is slightly skewed to the right, indicating that most of the radius_mean values are concentrated in the lower range.

- This distribution helps to understand the variance of the independent variable in the dataset.

5. Density plot of texture_mean:

- Purpose:
 - This plot shows the distribution of texture_mean values in the training dataset.
- Notes:
 - The density curve is slightly skewed to the right, with most of the values concentrated in the lower range.
 - This provides insight into the variance of the dependent variable and highlights potential issues such as outliers or nonlinearity.

EVALUATE THE MODEL RMSE METRIC:

```
>
> # Predict on test data
> predictions <- predict(lm_model, newdata = test_data)
>
> # Calculate RMSE
> rmse <- sqrt(mean((test_data$texture_mean - predictions)^2))
> cat("Root Mean Squared Error (RMSE):", rmse, "\n")
Root Mean Squared Error (RMSE): 4.631138
>
```

▪ Model Prediction on Test Data:

Predictions were made using a linear regression model on the test dataset.

Predicted values of texture_mean were calculated using the fitted model.

▪ Model Evaluation using RMSE:

Root Mean Square Error (RMSE): 4.631138

This indicates that on average, the predicted values are about 4.63 units away from the observed texture_mean values in the test data.

A lower RMSE value indicates better model performance. However, given the range of texture_mean, this value indicates that there is room for improvement.

- **Finally the dataset used: Breast Cancer Wisconsin Dataset from Kaggle.**